

Содержание

1. Цель работы	3
2. Теоретическое введение	3
2.1. Многослойный перцептрон	3
2.1.1. Соглашение о числе слоёв	3
2.2. Функции активации	4
2.2.1. Сигмоида	4
2.2.2. Гиперболический тангенс	4
2.2.3. ReLU	4
2.3. Функции ошибки	4
2.3.1. Для регрессии — MSE	4
2.3.2. Для классификации — кросс-энтропия	4
2.4. Обучение методом Adam	5
2.5. Ранняя остановка	5
2.6. Стандартизация	5
2.7. Кросс-валидация	5
3. Программная реализация	6
3.1. Построение MLP	6
3.2. Вычислительный граф MLP	6
3.3. Обучение модели	7
3.4. Генерация данных	8
3.4.1. Регрессия	8
3.4.2. Классификация	8
3.5. Метрики	9
3.5.1. MSE (регрессия)	9
3.5.2. Accuracy (классификация)	9
3.5.3. Confusion matrix	9
4. Часть I. Регрессия на выборках из задания 1	9
4.1. Описание эксперимента	9
4.2. Сводные таблицы Test MSE	10
4.2.1. Полином + равномерный шум	10
4.2.2. $x \sin(2\pi x)$ + нормальный шум	10
4.3. Визуализация лучших моделей	10
4.4. Демонстрация переобучения	11
5. Часть II. Классификация на выборках из задания 2	11
5.1. Описание эксперимента	11
5.2. Визуализация датасетов	12
5.3. Сводные таблицы Test Accuracy	12

5.4. Визуализация лучших моделей	12
6. Оценка минимального размера обучающей выборки	12
6.1. Постановка задачи	12
6.2. Реализация	13
6.3. Результаты	14
7. K-fold кросс-валидация	14
7.1. Постановка задачи	14
7.2. Реализация	14
7.3. Результаты	15
8. Выводы	15
9. Структура проекта	16
10. Репозиторий	16

1. Цель работы

Цель лабораторной работы — реализация многослойного перцептрона (МСП) средствами библиотеки `torch.nn` и исследование его возможностей при решении задач регрессии и классификации.

В работе исследуются:

- три функции активации (Sigmoid, Tanh, ReLU);
- архитектуры с числом линейных слоёв от 2 до 4;
- число нейронов в скрытом слое от 1 до 5;
- регрессия на выборках из задания 1 (полином 3-й степени и осциллирующая функция $x \sin(2\pi x)$);
- классификация на выборках из задания 2 (Circles, XOR, Blobs, Spiral);
- оценка минимального размера обучающей выборки для достижения ассурасы $\geq 90\%$;
- K-fold кросс-валидация для подбора функции активации и числа слоёв.

2. Теоретическое введение

2.1. Многослойный перцептрон

Многослойный перцептрон (МСП, MLP — Multilayer Perceptron) — нейронная сеть прямого распространения, состоящая из входного, одного или нескольких скрытых и выходного слоёв.

Каждый слой l выполняет преобразование:

$$h^{(l)} = \varphi(W^{(l)}h^{(l-1)} + b^{(l)}),$$

где $W^{(l)}$ — матрица весов, $b^{(l)}$ — вектор смещений, φ — функция активации, $h^{(0)} = x$ — вход сети.

2.1.1. Соглашение о числе слоёв

Под числом слоёв понимается число **линейных** слоёв вместе с выходным. Таким образом:

- 2 слоя = 1 скрытый слой;
- 3 слоя = 2 скрытых слоя;
- 4 слоя = 3 скрытых слоя.

2.2. Функции активации

Нелинейная функция активации вводится после каждого линейного преобразования скрытого слоя. В работе рассматриваются три варианта.

2.2.1. Сигмоида

$$\sigma(a) = \frac{1}{1 + e^{-a}}, \quad \sigma(a) \in (0, 1).$$

Гладкая, монотонная, с насыщением при $|a| \rightarrow \infty$. Может приводить к затуханию градиента при большой глубине сети.

2.2.2. Гиперболический тангенс

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}}, \quad \tanh(a) \in (-1, 1).$$

Центрирован относительно нуля, что обычно ускоряет сходимость по сравнению с сигмойдой, но также подвержен затуханию градиента.

2.2.3. ReLU

$$\text{ReLU}(a) = \max(0, a).$$

Вычислительно эффективна, не имеет насыщения при $a > 0$. Градиент равен нулю при $a < 0$ (проблема «мёртвых нейронов»).

Sigmoid

Выход: $(0, 1)$

Гладкая, насыщенная

Затухание градиента

Tanh

Выход: $(-1, 1)$

Центрирована в нуле

Затухание градиента

ReLU

Выход: $[0, +\infty)$

Быстрая, без насыщения

«Мёртвые нейроны»

2.3. Функции ошибки

2.3.1. Для регрессии — MSE

$$E(w) = \frac{1}{N} \sum_{i=1}^N \left(\tilde{f}(w, x^{(i)}) - y^{(i)} \right)^2,$$

где $\tilde{f}(w, x)$ — выход сети с параметрами w .

2.3.2. Для классификации — кросс-энтропия

При использовании двух выходных нейронов и softmax:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=0}^1 y_c^{(i)} \log \hat{y}_c^{(i)},$$

где $\hat{y}_c = \text{softmax}(z)_c = \frac{e^{z_c}}{e^{z_0} + e^{z_1}}$ — предсказанная вероятность класса c .

2.4. Обучение методом Adam

Для оптимизации используется алгоритм Adam — адаптивный метод градиентного спуска с экспоненциально взвешенными средними первого и второго моментов градиента:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon},$$

где $\hat{m}_t, \hat{v}_t$ — скорректированные оценки, η — шаг обучения.

2.5. Ранняя остановка

В процессе обучения на каждой эпохе вычисляются потери на обучающей и тестовой частях. Сохраняется состояние модели с наименьшей тестовой ошибкой. Это позволяет предотвратить переобучение: модель запоминает лучшую обобщающую способность.

2.6. Стандартизация

Перед обучением признаки и (для регрессии) целевые значения стандартизируются:

$$\hat{x} = \frac{x - \mu}{\sigma},$$

где μ и σ вычисляются по обучающей части. Стандартизация ускоряет сходимость и улучшает устойчивость обучения.

2.7. Кросс-валидация

Выборка S случайно делится на K равных подмножеств $S_1, \dots, S_K$. На каждой итерации i :

- обучающая выборка: $S'_i = \bigcup_{j \neq i} S_j$;
- тестовая выборка: $T'_i = S_i$.

Затем K результатов усредняются. Выбирается модель с наибольшей средней ассигасу.

3. Программная реализация

3.1. Построение MLP

В PyTorch нейросеть строится как последовательность модулей `nn.Sequential`. Каждый скрытый слой состоит из линейного преобразования `nn.Linear` и функции активации.

Исходный файл: mlp.py

```
ACTIVATIONS = {
    'sigmoid': nn.Sigmoid,
    'tanh':    nn.Tanh,
    'relu':    nn.ReLU,
}

def build_mlp(input_dim, output_dim, hidden_layers, hidden_dim, activation):
    layers = []
    in_f = input_dim
    act_cls = ACTIVATIONS[activation]

    for _ in range(hidden_layers):
        layers.append(nn.Linear(in_f, hidden_dim))
        layers.append(act_cls())
        in_f = hidden_dim

    layers.append(nn.Linear(in_f, output_dim))
    return nn.Sequential(*layers).to(device)
```

Параметры:

- `input_dim` — размерность входа (1 для регрессии, 2 для классификации);
- `output_dim` — размерность выхода (1 для регрессии, 2 для классификации);
- `hidden_layers` — количество скрытых слоёв (1, 2 или 3);
- `hidden_dim` — число нейронов в каждом скрытом слое (от 1 до 5);
- `activation` — функция активации (`sigmoid`, `tanh`, `relu`).

3.2. Вычислительный граф MLP

Для сети с двумя скрытыми слоями (3 линейных слоя) вычислительный граф имеет следующий вид:

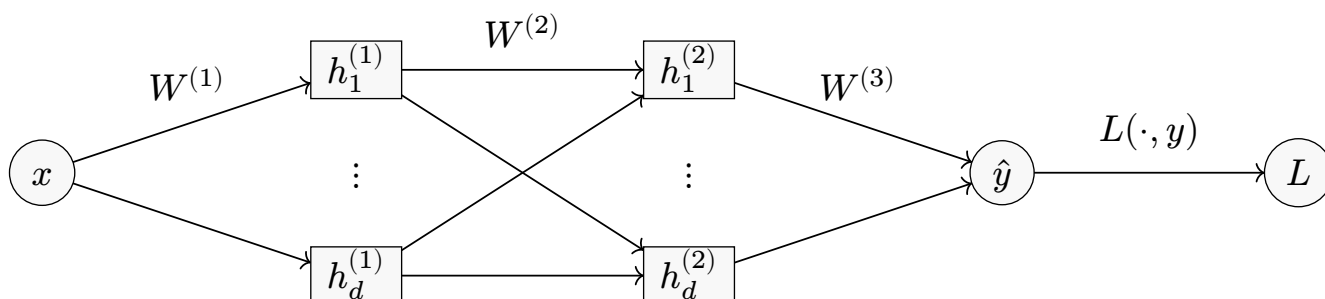


Рис. 1. Вычислительный граф MLP с двумя скрытыми слоями. Каждый скрытый узел включает линейное преобразование и функцию активации φ .

3.3. Обучение модели

Функция `train_model` реализует универсальное обучение: для регрессии используется `MSELoss`, для классификации — `CrossEntropyLoss`. На каждой эпохе сохраняется лучшее состояние модели по `val loss`.

Исходный файл: `mlr.py`

```
def train_model(model, X_train, y_train, X_val, y_val,
                 task='regression', epochs=300, lr=0.01):
    loss_fn = nn.MSELoss() if task == 'regression' else nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    history = {'train_loss': [], 'val_loss': []}
    best_state = None
    best_val = float('inf')

    for _ in range(epochs):
        model.train()
        optimizer.zero_grad()
        out = model(X_train)
        loss = loss_fn(out, y_train)
        loss.backward()
        optimizer.step()

        model.eval()
        with torch.no_grad():
            val_out = model(X_val)
            val_loss = loss_fn(val_out, y_val)

        history['train_loss'].append(float(loss.item()))
        history['val_loss'].append(float(val_loss.item()))

        if val_loss.item() < best_val:
            best_val = float(val_loss.item())
            best_state = copy.deepcopy(model.state_dict())

    if best_state is not None:
        model.load_state_dict(best_state)
    return history
```

3.4. Генерация данных

3.4.1. Регрессия

Используются две функции из задания 1:

1. Случайный полином 3-й степени: $f(x) = ax^3 + bx^2 + cx + d$.
2. Осциллирующая функция: $f(x) = x \sin(2\pi x)$.

Для полинома добавляется равномерный шум, для тригонометрической функции — нормальный.

Исходный файл: generators.py

```
def random_polynomial(seed=SEED):
    rng = np.random.default_rng(seed)
    a, b, c, d = rng.uniform(-3, 3, 4)
    def f(x):
        return a * x**3 + b * x**2 + c * x + d
    return f, (a, b, c, d)

def trig_function():
    def f(x):
        return x * np.sin(2 * np.pi * x)
    return f

def generate_regression_sample(f, n, eps0, noise_mode='uniform',
                              sigma=None, seed=SEED):
    rng = np.random.default_rng(seed)
    x = rng.uniform(-1, 1, n)
    noise = generate_noise(n, eps0, mode=noise_mode, sigma=sigma, seed=seed+1)
    y = f(x) + noise
    return x.reshape(-1, 1), y.reshape(-1, 1)
```

3.4.2. Классификация

Используются четыре типа выборок из задания 2: Circles, XOR, Blobs, Spiral.

```
CLS_DATASETS = [
    ('Circles', make_circles),
    ('XOR',     make_xor),
    ('Blobs',   make_blobs),
    ('Spiral',  make_spiral),
]
```


3.5. Метрики

3.5.1. MSE (регрессия)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2.$$

3.5.2. Accuracy (классификация)

$$\text{accuracy} = \frac{T_P + T_N}{T_P + F_P + T_N + F_N}.$$

3.5.3. Confusion matrix

$$\begin{pmatrix} T_N & F_P \\ F_N & T_P \end{pmatrix}$$

```
def classification_metrics(y_true, y_pred):
    y_true = np.asarray(y_true).ravel()
    y_pred = np.asarray(y_pred).ravel()
    tn = int(np.sum((y_true == 0) & (y_pred == 0)))
    tp = int(np.sum((y_true == 1) & (y_pred == 1)))
    fp = int(np.sum((y_true == 0) & (y_pred == 1)))
    fn = int(np.sum((y_true == 1) & (y_pred == 0)))
    acc = (tp + tn) / max(tp + tn + fp + fn, 1)
    return {'accuracy': float(acc), 'confusion_matrix': np.array([[tn, fp], [fn, tp]])}
```

4. Часть I. Регрессия на выборках из задания 1

4.1. Описание эксперимента

Для каждой из двух функций (полином и $x \sin(2\pi x)$) строится MLP с перебором всех комбинаций:

- функция активации: `sigmoid`, `tanh`, `relu`;
- число скрытых слоёв: 1, 2, 3 (линейных слоёв: 2, 3, 4);
- число нейронов в скрытом слое: 1, 2, 3, 4, 5.

Параметры обучения:

- $N = 280$ объектов, разбиение 75/25;
- `epochs = 350`, `learning rate = 0.02`;
- оптимизатор `Adam`, ранняя остановка по `val loss`.

Всего $3 \times 3 \times 5 = 45$ архитектур на каждую функцию.

4.2. Сводные таблицы Test MSE

В таблицах строки — число линейных слоёв (2, 3, 4), столбцы — число нейронов в скрытом слое ($N = 1, \dots, 5$). Значения — Test MSE.

4.2.1. Полином + равномерный шум

Активация	Слоёв	N=1	N=2	N=3	N=4	N=5
Sigmoid	2					
Sigmoid	3					
Sigmoid	4					
Tanh	2					
Tanh	3					
Tanh	4					
ReLU	2					
ReLU	3					
ReLU	4					

Таблица 1. Test MSE для полинома 3-й степени с равномерным шумом. Значения заполняются по результатам запуска.

4.2.2. $x \sin(2\pi x)$ + нормальный шум

Активация	Слоёв	N=1	N=2	N=3	N=4	N=5
Sigmoid	2					
Sigmoid	3					
Sigmoid	4					
Tanh	2					
Tanh	3					
Tanh	4					
ReLU	2					
ReLU	3					
ReLU	4					

Таблица 2. Test MSE для $x \sin(2\pi x)$ с нормальным шумом. Значения заполняются по результатам запуска.

4.3. Визуализация лучших моделей

Для лучшей модели по каждой функции активации строятся:

- график train/test loss по эпохам;
- график истинной функции и предсказания сети.

4.4. Демонстрация переобучения

Для наглядной демонстрации переобучения берётся:

- осциллирующая функция $f(x) = x \sin(2\pi x)$;
- маленькая обучающая подвыборка ($n_{\text{train}} = 10$);
- избыточная сеть (8 скрытых слоёв, 5 нейронов, tanh);
- обучение без ранней остановки (5000 эпох).

Исходный файл: `block1_main.py` (функция `run_overfit_demo`)

```
def train_no_early_stop(model, X_train, y_train, X_val, y_val,
                        epochs=2500, lr=0.01):
    loss_fn = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {'train_loss': [], 'val_loss': []}

    for _ in range(epochs):
        model.train()
        optimizer.zero_grad()
        out = model(X_train)
        loss = loss_fn(out, y_train)
        loss.backward()
        optimizer.step()

        model.eval()
        with torch.no_grad():
            val_loss = loss_fn(model(X_val), y_val)

        history['train_loss'].append(float(loss.item()))
        history['val_loss'].append(float(val_loss.item()))
    return history
```

В результате train loss продолжает падать, а test loss в определённый момент начинает расти — классический признак переобучения.

5. Часть II. Классификация на выборках из задания 2

5.1. Описание эксперимента

Для каждого из четырёх датасетов (Circles, XOR, Blobs, Spiral) выполняется перебор архитектур аналогично регрессии:

- функция активации: `sigmoid`, `tanh`, `relu`;
- число скрытых слоёв: 1, 2, 3;
- число нейронов в скрытом слое: 1, 2, 3, 4, 5.

Параметры:

- $N = 500$ объектов, разбиение 75/25;
- epochs = 250, learning rate = 0.03;
- выходной слой — 2 нейрона, CrossEntropyLoss.

5.2. Визуализация датасетов

5.3. Сводные таблицы Test Accuracy

В таблицах строки — число линейных слоёв (2, 3, 4), столбцы — число нейронов ($N = 1, \dots, 5$). Значения — Test accuracy.

Датасет	Активация	N=1	N=2	N=3	N=4	N=5
Circles	Sigmoid					
Circles	Tanh					
Circles	ReLU					
XOR	Sigmoid					
XOR	Tanh					
XOR	ReLU					
Blobs	Sigmoid					
Blobs	Tanh					
Blobs	ReLU					
Spiral	Sigmoid					
Spiral	Tanh					
Spiral	ReLU					

Таблица 3. Test accuracy для классификации (при 4 линейных слоях). Значения заполняются по результатам запуска.

5.4. Визуализация лучших моделей

Для лучшей модели строятся:

- график train/test loss;
- граница решений (decision boundary);
- confusion matrix.

6. Оценка минимального размера обучающей выборки

6.1. Постановка задачи

Найти минимальное количество элементов в обучающей выборке, при котором классификатор достигает accuracy $\geq 90\%$ на тестовой выборке.

Фиксированные параметры:

- функция активации: `tanh`;
- число линейных слоёв: 4;
- число нейронов: 5;
- `epochs = 250`, `lr = 0.03`.

Доля обучающей части варьируется от 10% до 95% с шагом 5%.

6.2. Реализация

Исходный файл: `block2_main.py` (функция `estimate_min_train_size`)

```
def estimate_min_train_size(datasets, activation='tanh', linear_layers=4,
                           hidden_dim=5, target_acc=0.90,
                           epochs=CLS_EPOCHS, lr=CLS_LR):
    fracs = np.arange(0.10, 0.96, 0.05)
    summary = []

    for ds in datasets:
        name = ds['name']
        X, y = ds['data']
        best_frac = None
        for frac in fracs:
            test_ratio = 1.0 - float(frac)
            X_tr, X_te, y_tr, y_te = train_test_split(X, y,
                                                       test_ratio=test_ratio, seed=SEED)
            X_tr_s, X_te_s, _, _ = standardize(X_tr, X_te)

            model = build_mlp(2, 2, linear_layers - 1, hidden_dim, activation)
            train_model(model, ...)

            pred = predict_classes(model, to_tensor(X_te_s))
            acc = classification_metrics(y_te, pred)['accuracy']

            if best_frac is None and acc >= target_acc:
                best_frac = float(frac)

        summary.append({
            'dataset': name,
            'min_frac': best_frac,
            'min_train_size': int(round(best_frac * len(X)))
                               if best_frac else None,
        })
    return summary
```

6.3. Результаты

Датасет	Активация	Слоёв	Нейронов	Min frac	Min N
Circles	tanh	4	5		
XOR	tanh	4	5		
Blobs	tanh	4	5		
Spiral	tanh	4	5		

Таблица 4. Минимальный размер обучающей выборки для ассигасы $\geq 90\%$. Значения заполняются по результатам запуска.

7. K-fold кросс-валидация

7.1. Постановка задачи

Для каждого датасета из задания 2 определить наилучшие значения:

1. функции активации (sigmoid / tanh / ReLU);
2. числа линейных слоёв (2, 3 или 4).

Число нейронов фиксировано: $d = 5$. Используется 5-fold кросс-валидация.

7.2. Реализация

Исходный файл: `block2_main.py` (функция `cv_grid_search`)

```
def kfold_indices(n, k=5, seed=SEED):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(n)
    return np.array_split(idx, k)

def cv_grid_search(X, y, activations=('sigmoid', 'tanh', 'relu'),
                  layers_grid=(2, 3, 4), hidden_dim=5,
                  k=CV_K, epochs=CLS_EPOCHS, lr=CLS_LR):
    folds = kfold_indices(len(X), k=k)
    rows = []
    best_row = None

    for act in activations:
        for nl in layers_grid:
            fold_accs = []
            for fi in range(k):
                val_idx = folds[fi]
                tr_idx = np.concatenate([folds[j] for j in range(k) if j != fi])

                X_tr, X_val = X[tr_idx], X[val_idx]
                y_tr, y_val = y[tr_idx], y[val_idx]
                X_tr_s, X_val_s, _, _ = standardize(X_tr, X_val)
```

```

model = build_mlp(2, 2, nl - 1, hidden_dim, act)
train_model(model, ...)

pred = predict_classes(model, to_tensor(X_val_s))
fold_accs.append(
    classification_metrics(y_val, pred)['accuracy'])

row = {
    'activation': act, 'linear_layers': nl,
    'cv_mean_acc': float(np.mean(fold_accs)),
    'cv_std_acc': float(np.std(fold_accs, ddof=1)),
}
rows.append(row)
if best_row is None or row['cv_mean_acc'] > best_row['cv_mean_acc']:
    best_row = row

return rows, best_row

```

7.3. Результаты

Для каждого из $3 \times 3 = 9$ комбинаций (активация $\times$ слои) проводится 5 итераций кросс-валидации. Лучшая комбинация выбирается по средней ассурасу.

Датасет	Лучшая активация	Слоёв	Mean acc	Std acc
Circles				
XOR				
Blobs				
Spiral				

Таблица 5. Лучшие результаты 5-fold кросс-валидации. Значения заполняются по результатам запуска.

8. Выводы

По результатам экспериментов:

1. Многослойный перцептрон, реализованный средствами `torch.nn`, успешно решает как задачи регрессии, так и бинарной классификации.
2. Для регрессии MLP аппроксимирует полином 3-й степени и осциллирующую функцию $x \sin(2\pi x)$. Качество аппроксимации зависит от числа нейронов и глубины сети.
3. Функция активации влияет на скорость обучения и качество:
 - `tanh` и `relu` обычно дают лучшие результаты;
 - `sigmoid` медленнее сходится из-за затухания градиента.

4. Для простых задач классификации (Blobs) высокая точность достигается даже при минимальной архитектуре (2 слоя, 1 нейрон).
5. Для сложных задач (Spiral) критически важны глубина сети и число нейронов. Функция активации ReLU и Tanh показывают лучшие результаты.
6. Переобучение наглядно проявляется при малом размере обучающей выборки ($n = 10$) и избыточной сети: train loss падает, а test loss начинает расти.
7. Кросс-валидация позволяет надёжно выбрать оптимальную архитектуру модели, усредняя результаты по нескольким разбиениям.
8. Минимальный размер обучающей выборки для accuracy $\geq 90\%$ зависит от сложности геометрии данных: для Blobs достаточно малого числа объектов, для Spiral требуется значительно больше.

9. Структура проекта

```
SMGMO/  
├─ lab3/  
│   ├── main.typ          # отчёт (Typst)  
│   ├── config.py         # параметры экспериментов (через env-переменные)  
│   ├── generators.py     # генерация данных (регрессия + классификация)  
│   ├── mlp.py            # MLP: построение, обучение, предсказание, метрики  
│   ├── block1_main.py    # Блок 1: регрессия + переобучение  
│   ├── block2_main.py    # Блок 2: классификация + min N + кросс-валидация  
│   └─ gui_launcher.py    # GUI-лаунчер (Tkinter)
```

10. Репозиторий

Исходный код проекта доступен на GitHub: <https://github.com/Q1zin/SMGMO.git>